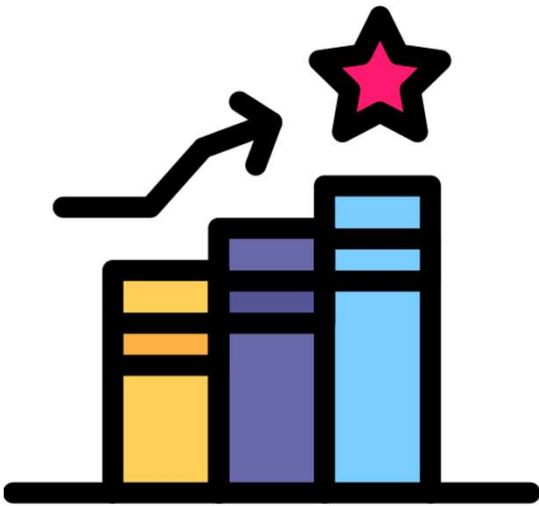


Carrera: Ingeniería de Software con IA

MÓDULOS Y PAQUETES PARA MACHINE LEARNING CON PYTHON



RESULTADO DE APRENDIZAJE:

Entender el funcionamiento interno de las estructuras de datos usadas en Machine Learning (memoria contigua, vectorización, representación matricial) y organizar un proyecto de ML en módulos, paquetes y librerías, reconociendo las implicaciones técnicas y éticas de las decisiones de preparación de datos.

CONTENIDO:

- ¿Dónde viven los datos?
- El motor de NumPy
- DataFrame a Matriz
- Tipos de datos
- Módulos y paquetes
- ¿Qué son las MLOps?
- ¿Donde conseguir datasets?
- Ética en la preparación de datos

¿Dónde viven los datos en la RAM ?

```
>>> data.shape  
(5000, 18)
```

Vive asignado de forma contigua en el **Heap**.

Es una referencia que vive en el **Stack** y apunta a la dirección del Heap donde inicia la matriz.

Saber dónde viven los datos permite optimizar el uso de RAM al trabajar con volúmenes masivos en modelos de aprendizaje.

EJEMPLO



¿Dónde viven los datos en la RAM ?

Imagina que estás en el área de sistemas del Banco de la Nación o de la cadena Plaza Vea:

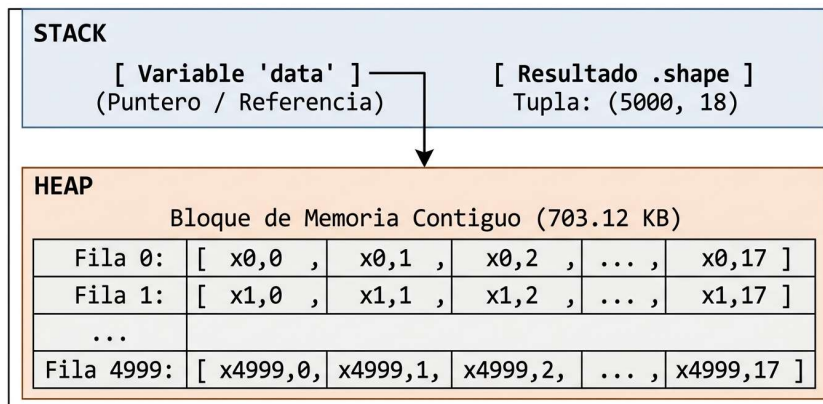
Tienes una tabla con **5,000 clientes o ventas** y **18 atributos** (DNI, Edad, Monto, Scoring crediticio, etc.).

Si cada celda guarda un número flotante de 64 bits (float64 = 8 bytes):

$$\text{Total de números} = 5000 \times 18 = 90,00 \text{ datos}$$

$$\text{Memoria consumida} = 90,000 \times 8 \text{ bytes} = 720,000 \text{ bytes} \approx 703.12 \text{ KB}$$

MEMORIA RAM VIRTUAL (Proceso de Python / Jupyter)



```
1 import numpy as np
2
3 # Generamos la matriz
4 data = np.zeros((5000, 18))
5
6 # Consultamos dimensiones y tipo
7 filas, columnas = data.shape
```

```
1 data.shape
(5000, 18)
```

```
1 data
array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]])
```

Memoria contigua y memoria dispersa

Memoria Contigua

Consiste en reservar un único bloque ininterrumpido de casilleros de memoria para almacenar un conjunto de datos. Cada elemento se ubica estrictamente al lado del anterior en direcciones consecutivas de memoria.



Se utiliza para procesar tablas de precios en bloques masivos de lectura rápida en memoria para aplicar descuentos masivos en milisegundos durante el *Cyber Days*.

Memoria dispersa

Consiste en almacenar los elementos de una estructura en diferentes ubicaciones o “bloques sueltos” de la memoria RAM.



Se utiliza en la cola de procesamiento de pedidos en línea, donde entran y salen miles de órdenes dinámicamente cada segundo sin saber de antemano cuántos pedidos habrá.

Memoria contigua y memoria dispersa

Lista de Python



Cada elemento vive en una celda distinta, conectada por punteros.

Array de NumPy



Un solo bloque continuo de memoria, como casilleros pegados.

En resumen

Acceder a memoria contigua es más rápido porque el procesador puede precargar datos vecinos (localidad de caché), en lugar de “saltar” de puntero en puntero.

Vectorización

La Vectorización es una técnica de programación que permite aplicar operaciones matemáticas o transformaciones a conjuntos completos de datos (vectores, matrices o DataFrames) de forma simultánea, eliminando la necesidad de escribir bucles for explícitos en Python.

Ejemplo:

Un sistema de cálculo de impuesto (IGV del 18%) para 1,000,000 de ventas registradas en un supermercado como **Plaza Vea**:

- **Sin Vectorización:** Python toma un precio del Heap, calcula el 18% en el Stack, guarda el resultado, busca la dirección del siguiente precio mediante punteros y repite el proceso 1,000,000 de veces.
- **Con Vectorización:** NumPy toma el bloque entero de **1,000,000** de precios almacenados de forma contigua en el Heap y le ordena al procesador (CPU) aplicar la operación a múltiples valores al mismo tiempo. (precios * 0.18 en NumPy/Pandas)

1000 operaciones/dato con for en Python en **horas**

1000 operaciones/dato vectorizadas en NumPy en **segundos**

Vectorización

Suma vectorizada

Tenemos $A = [2, 4, 6, 8]$

Realizamos: $A + 3$

La operación vectorizada significa que el **3** se aplica a **cada elemento**:

Por tanto:

$[5, 7, 9, 11]$

¿Cuántas operaciones se realizaron?

Para sumar dos vectores, deben tener **la misma cantidad de elementos**.

Sean: $A = [2, 4, 6]$

$B = [1, 3, 5]$

Se suman los elementos que ocupan la misma posición:

$$A + B = [2+1, 4+3, 6+5]$$

Por tanto:

$$A + B = [3, 7, 11]$$

Para la **RESTA**, al igual que en la suma, los vectores deben tener **la misma cantidad de elementos**.

Vectorización

Multiplicación por un escalar

Un escalar es simplemente un número.

Tenemos: $A = [2, 4, 6, 8]$

Multiplicamos por: $k = 3$

Entonces:

$$3A = [3(2), 3(4), 3(6), 3(8)]$$

Como resultado tendremos

$$[6, 12, 18, 24]$$

El escalar multiplica a todos los elementos del vector

Vectorización

Comparación vectorizada

Tenemos: $A = [8, 12, 15, 7, 20, 10]$

Queremos determinar cuáles valores cumplen $A \geq 12$

Para eso evaluamos cada posición:

Valor	≥ 12
8	Falso
12	Verdadero
15	Verdadero
7	Falso
20	Verdadero
10	Falso

¿Qué tipo de dato es ?

Por tanto: $[False, True, True, False, True, False]$

| Vectorización



Ejercicios:

Desarrollo de ejercicios

Vista y copia

Vista

Es una “ventana” o puntero alternativo que observa los **mismos datos originales** almacenados en el Heap. Si modificas el contenido de la vista, **los datos originales cambian automáticamente**, ya que no se reservó nuevo espacio en memoria.

Copia

Es un **objeto completamente nuevo** reservado en una dirección distinta del Heap. Duplica los datos en la RAM, por lo que modificar la copia **no altera** el objeto original.

```
1 import numpy as np
2
3 # Datos originales en el Heap
4 ventas = np.array([100, 200, 300, 400])
5
6 # VISTA: solo hace referencia a las ventas originales
7 vista_ventas = ventas[0:2]
8 vista_ventas[0] = 999 # Modifica también 'ventas'
9
10 # COPIA: crea un bloque de datos independiente en el Heap
11 copia_ventas = ventas[0:2].copy()
12 copia_ventas[1] = 888 # NO afecta a 'ventas'
```

Vista vs copia

Se desprende directamente del Concepto 1: si un array vive en un bloque continuo, un slice no necesita duplicar nada.

VISTA

Comparte memoria con el array original. Modificarla modifica el original. Prácticamente gratis: no copia datos.

COPIA

Reserva un bloque de memoria nuevo e independiente. No afecta al original, pero cuesta tiempo y RAM proporcional al tamaño de los datos.

Memoria perezosa (Lazy Memory) y SettingWithCopyWarning

Cuando tú tomas un puñado de maíz (por ejemplo, filtras solo los granos amarillos), Pandas tiene dos formas de dártelo:

Opción A (copia física): Agarrar una pala, trasladar los granos a un **cubo nuevo** y dártelo. Esto gasta tiempo y esfuerzo (memoria RAM).

Opción B (memoria perezosa): Pegar una nota adhesiva en el silo que diga: “*Los granos que te interesan están desde la fila 10 hasta la 50*”. No mueves ni un solo grano. Le dices al computador: “*No hagas el trabajo pesado todavía; solo recuerda dónde están*”.

Pandas no crea un cubo nuevo a menos que sea estrictamente necesario. Solo te da una “**vista**” (view) de los datos originales para ahorrar memoria y ser más rápido. Es perezoso porque evita copiar el maíz hasta el último momento.

Memoria perezosa (Lazy Memory) y SettingWithCopyWarning

Tú tienes tu notita adhesiva (la vista perezosa) que apunta a los granos dentro del **lugar original**.

Ahora decides: *“Voy a coger estos granos y los voy a pintar de color azul”*.

Cuando intentas hacer eso en Python (asignar un valor nuevo a esa porción de datos), Pandas se pone nervioso y te grita:

¡Oye, cuidado! No sé si quieres pintar los granos DENTRO del lugar original, o si quieres pintar solo los granos de tu cubo imaginario.

¡Dime explícitamente qué quieres hacer!

Memoria perezosa (Lazy Memory) y SettingWithCopyWarning

MEMORIA PEREZOSA

NumPy y Pandas, construido sobre NumPy, evita copiar datos hasta que es estrictamente necesario. Un slice simple casi siempre es vista; una selección compleja (índices no contiguos, condiciones booleanas) casi siempre fuerza una copia.

ES IMPORTANTE EN LA PRÁCTICA... PORQUE ?

Esta es la causa técnica detrás del SettingWithCopyWarning de Pandas: al seleccionar un subconjunto y modificarlo, Pandas a veces no puede garantizar si trabajas sobre una vista o una copia, una fuente real de bugs silenciosos, no solo una advertencia molesta.

¿Por qué el “Big Data” en Machine Learning necesitaría vectorización y no simplemente una computadora más rápida?

En **Big Data**, la diferencia entre tener una computadora más rápida y usar vectorización no es solo cuestión de tiempo: es la frontera entre que un modelo sea ejecutable o técnicamente imposible.

Si un modelo de Machine Learning procesa **100 terabytes** de datos elemento por elemento (secuencialmente), una CPU el doble de rápida tardará 50 horas en lugar de 100. Sigue siendo ineficiente.

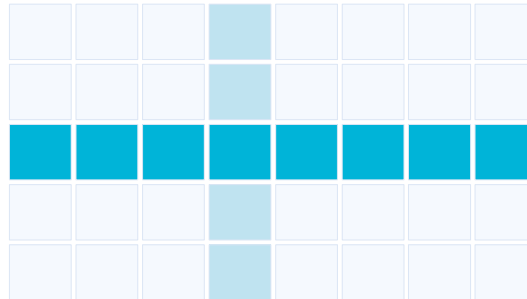
En Big Data, el problema real casi nunca es qué tan rápido calcula la CPU, sino **qué tan rápido se mueven los datos desde la RAM/Disco hasta el procesador.**

¿Por qué el “Big Data” en Machine Learning necesitaría vectorización y no simplemente una computadora más rápida?

En Big Data y ML, la velocidad (vectorización) es tu mejor amiga, pero la inmutabilidad controlada (usar copias cuando toca) es tu chaleco antibalas.

La `SettingWithCopyWarning` no es una molestia; es un salvavidas que te dice: “*¡Para! ¿Estás seguro de que quieres dispararle al lugar original?*”

Dimensionalidad: filas y columnas como vectores



fila = un cliente

columna = una característica (sueldo)

MATRIZ (8000, 16)

8000 filas (clientes) × 16 columnas (características). Cada fila es un vector fila; cada columna, un vector columna.

data.shape

Saltamos a Machine Learning

- Scikit-learn no recibe DataFrames directamente: recibe matrices de NumPy (arreglos bidimensionales, sin nombres ni índices).
- El algoritmo matemático (ejemplo: regresión lineal) calcula $Y = X \cdot \theta$ (producto punto), definido para matrices, no para tablas con etiquetas.

```
data['INGRESO'].values      # vector columna  
data[['INGRESO','EDAD']].values  # matriz de 2 columnas
```

El DataFrame es una capa de conveniencia (nombres, índices) construida sobre arrays de NumPy; .values la descarta.

| Loc, iloc: cómo se extrae la matriz

Usa **iloc** cuando sepas exactamente el número de filas que quieres (por ejemplo, separar los primeros 1000 datos para prueba).

Usa **loc** cuando busques por fecha o ID (`df.loc[df['ID'] == 123]`). Y si vas a modificar lo que extraes, pregúntate siempre: *“¿Esto es un trozo contiguo del silo?”*. Si lo es, ponle un `.copy()` al final para comprar tu propio terreno y evitar que el `SettingWithCopyWarning` te arruine el día.

Loc, iloc: cómo se extrae la matriz

	.loc[]	.iloc[]
Selecciona por	Etiqueta (nombres)	Posición entera
Ejemplo	<code>data.loc[:, ['INGRESO','EDAD']]</code>	<code>data.iloc[:, [3, 5]]</code>
Riesgo	Resistente a reordenar columnas	Frágil si el orden cambia (merge, drop)

Entonces:

El resultado de **.loc** o **.iloc** puede ser vista o copia, igual que un slice de **NumPy**. Por eso, al preparar **X** para Scikit-learn, se encadena con **.values** (o **.to_numpy()**): fuerza una copia independiente, dejando claro que **X** ya no comparte memoria con el DataFrame original.

Memoria(Silo) y dtypes

Imagina tu RAM (memoria) como una estantería gigante con **huecos de tamaño fijo**.

- Cada hueco es un “byte” (8 bits).
- El **dtype** define **cuántos huecos consecutivos**.

Tipo (Caja)	Tamaño	Su uso
int8	1 byte	Números del -128 al 127
int64	8 bytes	Números enormes (hasta 9 trillones)
float32	4 bytes	Decimales con precisión media
float64	8 bytes	Decimales con precisión milimétrica
object	(8 bytes + puntero)	Texto (strings) o mezcla de cosas

Memoria y dtypes

Downcasting

Bajar el tamaño de la caja

Es la conversión de un tipo de dato a otro de **menor precisión o capacidad** (por ejemplo, pasar un flotante de 64 bits a 32 bits, o de un entero de 64 bits a un entero de 8 bits).

Tipo	Tamaño / comportamiento
int64	8 bytes, tamaño fijo
float64	8 bytes, tamaño fijo
object (string)	Tamaño variable; guarda punteros a texto → mayor consumo de RAM

Hacer downcasting sin verificar los datos puede corromper completamente la información

Para hacer un Downcasting se utilizan funciones automáticas que inspeccionan el valor mínimo y máximo de la columna para elegir el menor tipo de dato seguro.

Los valores faltantes NaN



Representa datos ausentes o nulos en análisis de datos y Machine Learning.

La mayoría de modelos de Machine Learning (como regresiones o redes neuronales) no pueden procesar NaN y lanzarán un error al intentar entrenar.

En operaciones matemáticas, cualquier número operado con NaN da como resultado NaN

Un NaN no es igual a nada, ni siquiera a sí mismo (`np.nan == np.nan` devuelve False).

Los valores faltantes NaN

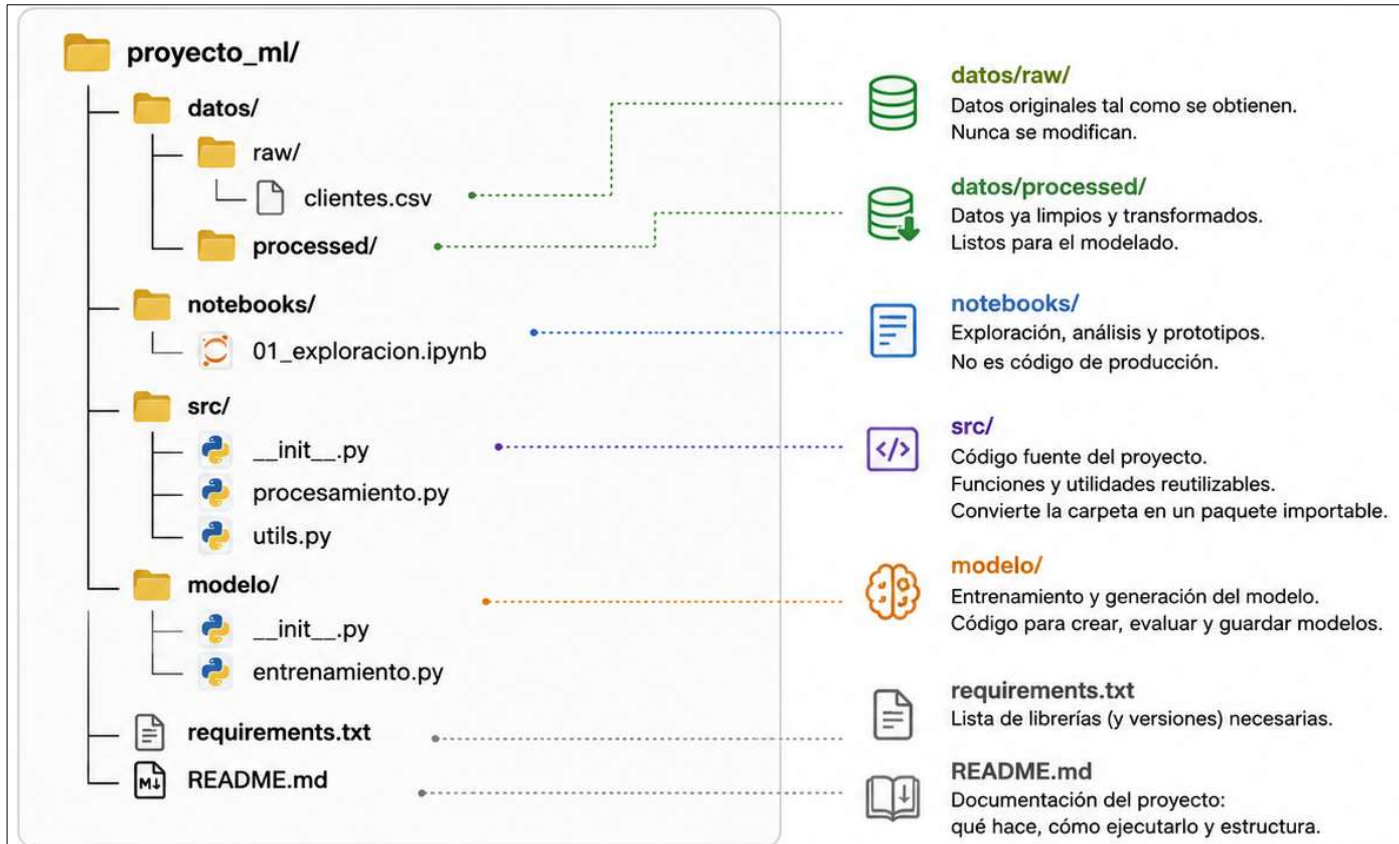
Representa datos ausentes o nulos en análisis de datos y Machine Learning.

¿Cómo se gestionan?

Estrategia	Método	Cuándo usarlo
Eliminación	df.dropna()	Cuando los NaN son muy pocos (< 5%) y no afectarán la muestra.
Imputación	df.fillna() / SimpleImputer	Rellenar con la media/mediana (numéricos) o la moda (categóricos).
Modelos nativos	Algoritmos como XGBoost / LightGBM	Tolera los NaN automáticamente sin necesidad de rellenarlos.

Imputar no es “recuperar el dato real” es una decisión de modelado

Arquitectura de un proyecto real en ML



Arquitectura de un proyecto real en ML



Buenas prácticas

- ✓ No modificar datos en raw/
- ✓ Todo código reutilizable en src/
- ✓ Entorno reproducible con requirements.txt
- ✓ Documentar siempre en README.md
- ✓ Usar control de versiones (Git)

Fuentes de datos para proyectos de Machine Learning



Kaggle

Datasets preparados para Data Science y Machine Learning.

Clasificación, regresión, series temporales, imágenes, NLP, salud, finanzas, marketing

NOAA / National Centers for Environmental Information

Datos climáticos y ambientales.

temperatura, precipitación, Océanos, atmósfera, eventos meteorológicos

Google Trends

Analiza tendencias de búsqueda.

Términos de búsqueda, evolución temporal, países/regiones, Categorías, búsquedas web, YouTube, imágenes

World Health Organization — WHO

datos relacionados con salud pública.

mortalidad, enfermedades, vacunas, sistemas de salud, indicadores sanitarios

Fuentes de datos para proyectos de Machine Learning

Data.gov.sg

Usa datos de Singapur

NASA Earthdata

Datos científicos y ambientales

AWS Open data

Grandes conjuntos de datos públicos

Pew research center

Datos sociales y demográficos

¿Cómo elegir un dataset?

Criterio	Pregunta
Problema	¿Qué problema quiero resolver?
Variables	¿Qué características contiene?
Tamaño	¿Tiene suficientes registros?
Calidad	¿Tiene muchos valores faltantes?
Variable objetivo	¿Qué quiero predecir?
Tipo de problema	¿Clasificación, regresión o clustering?
Licencia	¿Puedo utilizar los datos?
Actualidad	¿Los datos siguen siendo relevantes?
Sesgo	¿Existe algún grupo subrepresentado?
Ética	¿El uso de los datos puede generar riesgos?

¿Qué son las MLOps?

Es un conjunto de **prácticas, procesos y herramientas** que permiten llevar un modelo de Machine Learning desde la experimentación hasta un entorno donde pueda ser **reproducido, desplegado, monitoreado y mantenido**.

MLOps = Machine Learning Operations

Es la disciplina que une el desarrollo de modelos de Machine Learning con las operaciones de software tradicional (DevOps). Su objetivo es hacer que un modelo pase de ser un experimento en un notebook a un servicio estable, automático y escalable en producción.

Ciclo continuo de MLOps

Su objetivo fundamental es resolver el problema de la **deuda técnica en ML**, garantizando que los modelos mantengan su rendimiento predictivo a lo largo del tiempo en entornos dinámicos. Este ciclo se articula en torno a tres principios de entrega continua: **Integración Continua (CI)**, **Entrega/Despliegue Continuo (CD)** y **Entrenamiento Continuo (CT)**.

El Ciclo Continuo de MLOps es, en esencia, un sistema de control de retroalimentación negativa aplicado al aprendizaje automático. Su propósito es minimizar la entropía del sistema productivo, garantizando que el modelo se adapte a la evolución del entorno mediante la automatización total del flujo de vida, desde la ingesta de datos hasta la respuesta operativa.

Ciclo continuo de MLOps

El Ciclo Continuo de MLOps:

Del experimento al valor en producción



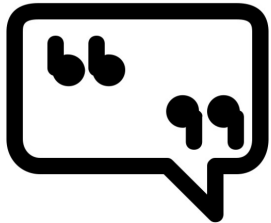
Ética en la preparación de datos

Las decisiones técnicas de hoy (imputar, eliminar, convertir tipos) no son neutrales: tienen consecuencias sobre a quién representa el modelo final.

La **ética en la preparación de datos** aborda la responsabilidad moral al recolectar, limpiar, transformar y seleccionar la información que alimentará a un modelo de Machine Learning. Dado que los algoritmos aprenden directamente de los datos, cualquier sesgo o manipulación en esta etapa se amplificará en las predicciones finales.

Ética en la preparación de datos

- **Sesgo de selección y representatividad:** Si el dataset excluye a un grupo demográfico específico o contiene muestras desproporcionadas, el modelo tomará decisiones sesgadas o discriminatorias en producción.
- **Privacidad y anonimización:** Limpiar datos no solo implica quitar nulos, sino eliminar Información de Identificación Personal (PII). Un downcasting o codificación deficiente puede dejar al descubierto identidades reales.
- **Fuga de datos (Data Leakage):** Incluir información del futuro o variable objetivo dentro del set de entrenamiento puede inflar falsamente el rendimiento del modelo, llevando a tomar decisiones basadas en resultados irrealistas.
- **Manipulación de imputación (NaN):** Rellenar valores faltantes con promedios o supuestos arbitrarios puede alterar la realidad subyacente de comunidades o fenómenos representados.
- **Transparencia y consentimiento:** Utilizar datos recolectados sin el consentimiento expreso de los usuarios o para un propósito distinto al originalmente pactado vulnera la confianza y los marcos regulatorios (como RGPD).



“Los sistemas de información no solo mueven datos... mueven ideas, decisiones y el futuro de las organizaciones”